

GitHub Practical Question Bank - Answers (Q.2)

22. Generate a Python program using GitHub Copilot to sort a list of numbers in ascending order.

Answer:

1. open Visual Studio Code
2. install GitHub Copilot extension from Extensions and sign in with GitHub
3. create new file sort_numbers.py
4. write comment : # Write a Python program to sort a list of numbers in ascending order
5. press Enter, Copilot suggests the code, press Tab to accept
6. save the file (Ctrl+S)
7. open terminal
8. `python sort_numbers.py`

23. Create a Python program using GitHub Copilot to calculate the factorial of a number.

Answer:

1. open Visual Studio Code
2. install GitHub Copilot extension from Extensions and sign in with GitHub
3. create new file factorial.py
4. write comment : # Write a Python program to calculate the factorial of a number
5. press Enter, Copilot suggests the code, press Tab to accept
6. save the file (Ctrl+S)
7. open terminal
8. `python factorial.py`

24. Perform the Git commit operation with an appropriate commit message, push the changes to GitHub, and verify that the remote repository has been updated successfully.

Answer:

git configuration :

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

1. `mkdir project24`
2. `cd project24`
3. `git init`
4. `git branch -M main`
5. `vim hello.txt`
6. go to github.com, click + (top right), click New repository, repository name project24, click Create repository, copy the repository link
7. `git remote add origin repo-link`
8. `git remote -v`
9. `git status`
10. `git add .`
11. `git commit -m "Updated project files"`
12. `git push -u origin main`
13. go to github repository, refresh the page, check the file and latest commit message

25. Create a new local Git repository named GitLab. Create the following files: student.py and marks.py. Add all files to the staging area and commit them with the message "Initial Project Structure".

Answer:

git configuration :

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

1. mkdir GitLab
2. cd GitLab
3. git init
4. vim student.py
5. vim marks.py
6. git status
7. git add .
8. git status
9. git commit -m "Initial Project Structure"
10. git log --oneline

26. Connect the local repository to a GitHub repository and push all files. Perform the following file management operations: Modify student.py. Create a new file named attendance.py.

Answer:

git configuration :

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

1. mkdir GitLab
2. cd GitLab
3. git init
4. git branch -M main
5. vim student.py
6. vim marks.py
7. git add .
8. git commit -m "Initial Project Structure"
9. go to github.com, click + (top right), click New repository, repository name GitLab, click Create repository, copy the repository link
10. git remote add origin repo-link
11. git remote -v
12. git push -u origin main
13. vim student.py
14. vim attendance.py
15. git status
16. git add .
17. git commit -m "Modified student.py and added attendance.py"
18. git push origin main
19. go to github repository, refresh the page, check student.py and attendance.py

27. Create an Issue titled "Create Student Management Module" and assign the enhancement label to it. Close the created Issue after marking it as completed.

Answer:

1. go to github.com, open the repository
2. click Issues tab
3. click New issue
4. title : Create Student Management Module
5. write a description
6. on the right side click Labels, select enhancement
7. click Submit new issue
8. open the created issue, click Close issue (Close as completed)
9. go to Issues tab, click Closed, check the issue is closed

28. Create an Issue titled "Create Student Management Module" and assign the enhancement label to it.

Answer:

1. go to github.com, open the repository
2. click Issues tab
3. click New issue
4. title : Create Student Management Module
5. write a description
6. on the right side click Labels, select enhancement
7. click Submit new issue
8. go to Issues tab, check the issue is open with enhancement label

29. Create a branch named development, switch to it, create a file named development.txt, add suitable content, and commit the changes. Switch back to the main branch and verify the branch status.

Answer:

git configuration :

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

1. mkdir project29
2. cd project29
3. git init
4. git branch -M main
5. vim README.txt
6. git add .
7. git commit -m "Initial commit"
8. git branch development
9. git switch development
10. vim development.txt
11. git status
12. git add development.txt
13. git commit -m "Added development.txt"
14. git switch main

15. git branch
16. git status

30. Create a branch named test, add a file named test.txt, commit the changes, switch back to the main branch, and merge the test branch into the main branch.

Answer:

git configuration :

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

1. mkdir project30
2. cd project30
3. git init
4. git branch -M main
5. vim README.txt
6. git add .
7. git commit -m "Initial commit"
8. git branch test
9. git switch test
10. vim test.txt
11. git add test.txt
12. git commit -m "Added test.txt"
13. git switch main
14. git merge test
15. ls
16. git log --oneline

31. Create an issue in the GitHub repository with the title "Add New Feature". Provide a suitable description and manage the issue by adding an appropriate label.

Answer:

1. go to github.com, open the repository
2. click Issues tab
3. click New issue
4. title : Add New Feature
5. description : Add a new feature to the student management module to export records to CSV
6. on the right side click Labels, select enhancement
7. click Submit new issue
8. go to Issues tab, check the issue with enhancement label

32. Create two branches named student-details and marks-details. Add student.py file to the student-details branch and marks.py file to the marks-details branch. Commit the changes and merge both branches into the main branch.

Answer:

git configuration :

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

1. mkdir project32

2. `cd project32`
3. `git init`
4. `git branch -M main`
5. `vim README.txt`
6. `git add .`
7. `git commit -m "Initial commit"`
8. `git branch student-details`
9. `git switch student-details`
10. `vim student.py`
11. `git add student.py`
12. `git commit -m "Added student.py in student-details branch"`
13. `git switch main`
14. `git branch marks-details`
15. `git switch marks-details`
16. `vim marks.py`
17. `git add marks.py`
18. `git commit -m "Added marks.py in marks-details branch"`
19. `git switch main`
20. `git merge student-details`
21. `git merge marks-details -m "Merged marks-details branch"`
22. `ls`
23. `git log --oneline --graph`

33. Modify the README.md file by adding installation steps for the project. Commit the changes, push the branch, create a Pull Request, and merge it into the main branch.

Answer:

git configuration :

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

1. `mkdir project33`
2. `cd project33`
3. `git init`
4. `git branch -M main`
5. `vim README.md`
6. `git add .`
7. `git commit -m "Initial commit"`
8. go to github.com, click + (top right), click New repository, repository name project33, click Create repository, copy the repository link
9. `git remote add origin repo-link`
10. `git remote -v`
11. `git push -u origin main`
12. `git switch -c feature-readme`
13. `vim README.md`
14. `git status`
15. `git add README.md`
16. `git commit -m "Added installation steps in README.md"`

17. `git push -u origin feature-readme`
18. go to github repository, click Compare & pull request for feature-readme
19. write title and description, click Create pull request
20. click Merge pull request, click Confirm merge
21. `git switch main`
22. `git pull origin main`
23. `git log --oneline`

34. Create a new file named commands.txt and write any five Git commands with their purpose. Commit the file with an appropriate message and push the changes to the remote repository and also implement git pull request.

Answer:

git configuration :

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

1. `mkdir project34`
2. `cd project34`
3. `git init`
4. `git branch -M main`
5. `vim README.md`
6. `git add .`
7. `git commit -m "Initial commit"`
8. go to github.com, click + (top right), click New repository, repository name project34, click Create repository, copy the repository link
9. `git remote add origin repo-link`
10. `git remote -v`
11. `git push -u origin main`
12. `git switch -c feature-commands`
13. `vim commands.txt`
14. write in file : `git init` - creates repository, `git add` - adds files to staging area, `git commit` - saves changes, `git push` - uploads to GitHub, `git pull` - downloads from GitHub
15. `git status`
16. `git add commands.txt`
17. `git commit -m "Added commands.txt with five Git commands"`
18. `git push -u origin feature-commands`
19. go to github repository, click Compare & pull request for feature-commands
20. write title and description, click Create pull request
21. click Merge pull request, click Confirm merge
22. `git switch main`
23. `git pull origin main`
24. `git log --oneline`

35. Create a Pull Request from the feature-readme branch to the main branch. Merge the Pull Request, switch to the main branch, and update the local repository with the latest changes.

Answer:

git configuration :

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

1. mkdir project35
2. cd project35
3. git init
4. git branch -M main
5. vim README.md
6. git add .
7. git commit -m "Initial commit"
8. go to github.com, click + (top right), click New repository, repository name project35, click Create repository, copy the repository link
9. git remote add origin repo-link
10. git remote -v
11. git push -u origin main
12. git switch -c feature-readme
13. vim README.md
14. git add README.md
15. git commit -m "Updated README.md"
16. git push -u origin feature-readme
17. go to github repository, click Compare & pull request for feature-readme
18. write title and description, click Create pull request
19. click Merge pull request, click Confirm merge
20. git switch main
21. git pull origin main
22. git log --oneline

36. Clone a GitHub repository to your local system. Create a new file named README.md, add your name and roll number and your project information, commit the changes with an appropriate commit message, and push the updated repository to GitHub.

Answer:

git configuration :

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

1. mkdir clonerepo
2. cd clonerepo
3. go to github repository, click Code, copy the repository link
4. git clone repo-link
5. cd repository-name
6. vim README.md
7. git status
8. git add README.md
9. git commit -m "Added README.md with name, roll number and project information"
10. git push origin main
11. go to github repository, refresh the page, check README.md

37. Create a branch named web-development. Add two files, index.html and style.css, to the branch. Commit the changes, push the branch to GitHub, and verify the branch on the GitHub repository.

Answer:

git configuration :

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

1. mkdir project37
2. cd project37
3. git init
4. git branch -M main
5. vim README.md
6. git add .
7. git commit -m "Initial commit"
8. go to github.com, click + (top right), click New repository, repository name project37, click Create repository, copy the repository link
9. git remote add origin repo-link
10. git remote -v
11. git push -u origin main
12. git switch -c web-development
13. vim index.html
14. vim style.css
15. git status
16. git add .
17. git commit -m "Added index.html and style.css in web-development branch"
18. git push -u origin web-development
19. go to github repository, click branch dropdown, check web-development branch and its files

38. Clone an existing GitHub repository to the local CentOS system. Create a new Python file named calculator.py and a text file named demo.txt. Display the repository status before and after staging the files.

Answer:

git configuration :

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

1. mkdir clonerepo
2. cd clonerepo
3. go to github repository, click Code, copy the repository link
4. git clone repo-link
5. cd repository-name
6. vim calculator.py
7. vim demo.txt
8. git status
9. git add .
10. git status

39. Create a new branch named feature in the Git repository. Switch to the feature branch, create a file named feature.txt, commit the changes, and display the available branches.

Answer:

git configuration :

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

1. mkdir project39
2. cd project39
3. git init
4. git branch -M main
5. vim README.txt
6. git add .
7. git commit -m "Initial commit"
8. git branch feature
9. git switch feature
10. vim feature.txt
11. git status
12. git add feature.txt
13. git commit -m "Added feature.txt in feature branch"
14. git branch